DIVISION OF PHYSICAL SCIENCES AND MATHEMATICS
College of Arts and Sciences | University of the Philippines Visayas
Miagao, Iloilo, 5023, Philippines | Tel. No. (033) 513-7020
Email Address: psm.upvisayas@up.edu.ph

COURSE MATERIAL

Unix Shell

CMSC 125 Lab 1

Prepared by: Rene Andre Bedonia Jocsing

Published: January 29, 2026 **Deadline:** May 21, 2026

This laboratory assignment focuses on process management and I/O redirection using the POSIX API in C. You will implement a simplified Unix shell that demonstrates how operating systems manage processes and handle basic I/O operations.

Background

A shell is a command-line interpreter that provides a user interface for interacting with the operating system. It reads commands from the user, interprets them, and executes them by creating and managing processes. Your shell will demonstrate fundamental OS concepts including process creation, program execution, and I/O redirection.

Learning Objectives

By the end of this laboratory exercise, students should be able to:

- Implement an interactive command-line interface
- Use the POSIX process API (`fork()`, `exec()` family, `wait()`/`waitpid()`)
- Implement I/O redirection using file descriptors and `dup2()`
- Handle background processes
- Parse and tokenize user input using C (recall: CMSC 124)
- Distinguish between built-in and external commands
- Follow modern C best practices and documentation standards

Task

Build a Unix shell (`mysh`) that supports:

- Interactive command execution with prompt
- Built-in commands: `exit`, `cd`, `pwd`
- External command execution via `fork()` and `exec()`
- I/O redirection: > (truncate), >> (append), < (input)
- Background execution: `&`

Required Features

Interactive Command Loop

Your shell must present an interactive prompt and process commands in a loop:

```
$ ./mysh
mysh> pwd
/home/user
mysh> cd /tmp
mysh> pwd
/tmp
mysh> ls -la
total 24
drwxrwxrwt 10 root root 4096 Dec  7 10:30 .
drwxr-xr-x 19 root root 4096 Oct 15 09:15 ..
...
mysh> echo "Hello, World!"
Hello, World!
mysh> nonexistent_command
mysh: command not found: nonexistent_command
mysh> exit
$
```

Built-in Commands

The following commands must be implemented directly in the shell (no forking):

- **exit**: Terminate the shell (ensure that zombie processes are terminated)
- **cd** : Change current working directory using `chdir()`
- **pwd**: Print current working directory using `getcwd()`

External Command Execution

All non-built-in commands must be executed by forking a child process:

1. Fork a child process using `fork()`
2. In child: execute command using `execvp()`
3. In parent: wait for child completion (unless background job)
4. Report exit status and errors appropriately

I/O Redirection

Support standard I/O redirection operators:

```
mysh> ls -la > output.txt
mysh> cat output.txt
total 24
drwxr-xr-x 2 user user 4096 Dec  7 10:30 .
...
mysh> wc -l < output.txt
      12
mysh> echo "appending text" >> output.txt
mysh> sort < unsorted.txt > sorted.txt
```

Implementation requirements:

- Use `open()` to open files with appropriate flags (`O_RDONLY`, `O_WRONLY`, `O_CREAT`, `O_TRUNC`, `O_APPEND`)
- Use `dup2()` to redirect file descriptors (stdin, stdout)

- Close file descriptors after use to prevent leaks
- Handle file errors gracefully (permission denied, file not found, etc.)
- Support combining input and output redirection: `sort < input.txt > output.txt`

Background Execution

Support background job execution with the `&` operator:

```
mysh> sleep 10 &
[1] Started background job: sleep 10 (PID: 12345)
mysh> echo "Shell still responsive"
Shell still responsive
mysh> sleep 20 &
[2] Started background job: sleep 20 (PID: 12346)
mysh> ls -la
(output appears immediately)
mysh>
```

Implementation requirements:

- Detect `&` at end of command line
- Parent does not wait for background process to complete
- Print job start message with job ID and PID
- Track background processes to avoid zombie processes
- Use `waitpid()` with `WNOHANG` to reap completed background jobs

Technical Requirements

Required System Calls

- **Process management:** `fork()`, `execvp()`, `wait()`, `waitpid()`, `getpid()`, `getppid()`
- **I/O:** `dup2()`, `open()`, `close()`
- **File system:** `chdir()`, `getcwd()`

Data Structures

Design appropriate data structures to represent commands. Consider:

```c
typedef struct {
    char *command;        // Command name
    char *args[256];      // Arguments (NULL-terminated)
    char *input_file;     // For < redirection (NULL if none)
    char *output_file;    // For > or >> redirection (NULL if none)
    bool append;          // true for >>, false for >
    bool background;      // true if & present
} Command;
```

Code Organization

Your implementation should follow good software engineering practices:

- Modular design with separate functions for distinct responsibilities
- Clear separation between parsing and execution
- Header files for data structures and function declarations
- Comprehensive error handling for all system calls

- Proper memory management (no memory leaks)
- Meaningful variable names and code comments

Implementation Notes

Command Parsing

Parse user input into tokens representing command, arguments, and operators:

- Use `strtok()` or similar to tokenize input by whitespace
- Detect special operators: >, >>, <, &
- Build command structure with extracted information
- Handle edge cases: multiple spaces, trailing whitespace, empty input

Process Execution

Execute commands by creating child processes:

```
pid_t pid = fork();

if (pid < 0) {
    perror("fork failed");
    return -1;
}

if (pid == 0) {  // Child process
    // Apply redirections if needed
    if (cmd.input_file) {
        int fd = open(cmd.input_file, O_RDONLY);
        if (fd < 0) {
            perror("open input file");
            exit(1);
        }
        dup2(fd, STDIN_FILENO);
        close(fd);
    }

    if (cmd.output_file) {
        int flags = O_WRONLY | O_CREAT;
        flags |= cmd.append ? O_APPEND : O_TRUNC;
        int fd = open(cmd.output_file, flags, 0644);
        if (fd < 0) {
            perror("open output file");
            exit(1);
        }
        dup2(fd, STDOUT_FILENO);
        close(fd);
    }

    // Execute command
    execvp(cmd.command, cmd.args);
    perror("exec failed");
    exit(127);
} else {  // Parent process
    if (!cmd.background) {
        int status;
        waitpid(pid, &status, 0);
        if (WIFEXITED(status)) {
            int exit_code = WEXITSTATUS(status);
```

```
            if (exit_code != 0) {
                printf("Command exited with code %d\n", exit_code);
            }
        }
    } else {
        printf("[%d] Started: %s (PID: %d)\n", job_id, cmd_str, pid);
        // Add to background job list
    }
}
```

File Descriptor Management

Properly manage file descriptors to avoid leaks:

- Close file descriptors after `dup2()` in child processes
- Remember: every `open()` creates an FD that must be closed

Background Process Management

Track background processes to prevent zombies:

- Maintain a list or array of background job PIDs
- Periodically call `waitpid(pid, &status, WNOHANG)` for each background job
- Remove completed jobs from the list
- Call this cleanup function at the start of each shell loop iteration

---

Common Pitfalls

- **Zombie processes:** Always reap child processes. Use `waitpid()` with `WNOHANG` for background jobs.
- **Built-in commands in child:** Built-ins like `cd` must execute in parent process, not child.
- **Memory leaks:** Free all dynamically allocated memory.
- **Parse errors:** Handle malformed input gracefully. Don't crash on missing arguments or files.
- **Redirection order:** Support both `< in > out` and `> out < in` orderings.

Testing Strategy

Test your shell thoroughly with diverse scenarios:

Basic Functionality

- External commands: `ls`, `echo`, `cat`, `grep`, `wc`
- Built-in commands: `cd`, `pwd`, `exit`
- Commands with multiple arguments: `ls -la /tmp`
- Nonexistent commands
- Empty input and whitespace-only input

I/O Redirection

- Output: `echo "test" > file.txt`
- Append: `echo "more" >> file.txt`
- Input: `wc -l < file.txt`
- Combined: `sort < input.txt > output.txt`
- Both orderings: `< in > out` and `> out < in`

- Nonexistent input file (should show error)
- Permission-denied output file (should show error)
- Overwriting existing files

Background Jobs

- Single background job: `sleep 10 &`
- Multiple concurrent background jobs
- Background job that exits immediately
- Background job with output redirection: `ls > out.txt &`
- Shell remains responsive while background jobs run

Edge Cases

- Very long command lines
- Commands with many arguments
- Rapid command sequences
- Background job completing before next command
- Redirection to `/dev/null`
- Spaces around redirection operators: `ls > file.txt`

Compare your shell's behavior with `bash` to verify correctness.

Deliverables

Your group's GitHub repository, in addition to a clean, incremental commit history showing individual commits, must contain:

1. All source files (`.c` and `.h` files)
2. `Makefile` with targets:
   - `all`: Compile the shell
   - `clean`: Remove binaries and object files
3. `README.md` with:
   - **Complete names** of group members.
   - Compilation and usage instructions
   - List of implemented features
   - Known limitations or bugs
   - Design decisions and architecture overview
   - Screenshots showing functionality
4. Test cases or test script demonstrating all required features

To submit, simply invite the instructor as a collaborator. Commits after the instructor has already graded the repository will not be considered.

Then, submit the following, **individually**, through email:

1. A short `reflection.txt` containing a brief summary of lessons learned and challenges encountered during the activity.
2. If in a group: a short `peer.txt` containing your thoughts about the work and conduct of your peers in the group during the activity (this is where you highlight issues, if any).
3. The link to your GitHub repository (the instructor will use this link to verify your submission)

Adhere to the following subject line: `[CMSC 125 Lab] Lab 1: Surname, Initials.`

For example: `[CMSC 125 Lab] Lab 1: Sanchez, SM.`

Academic Honesty

The usage of Large Language Models (e.g. ChatGPT, Claude, Deepseek, etc.) to generate code is considered cheating. As cheating is against the university's code of ethics, it is subject to failure in the course and harsh disciplinary action.

Important Dates

Progress reports and laboratory defense may be booked only during the dates and hours defined in the syllabus. It is mandatory to book appointments ahead of time on the course's booking page (also accessible on the course site).

Activity	Monday	Wednesday	Thursday
Week 1 Progress Report	Feb 2	Feb 4	Feb 5
Week 2 Progress Report	Feb 9	Feb 11	Feb 12
Week 3 Progress Report	Feb 16	Feb 18	Feb 19
Week 4 Laboratory Defense	**Feb 23**	**Feb 25**	**Feb 26**

The instructor must verify appointments ahead of time before they can be considered valid. See the syllabus for proper hours and more details.

Grading Rubric

Criteria	Excellent (90-100%)	Good (75-89%)	Fair (60-74%)	Poor (0-59%)
System Architecture (25%)	Modular design with clean separation of concerns; robust data structures; efficient resource usage.	Mostly modular; logic is functional but slightly coupled; data structures are appropriate.	Significant logic sprawl; monolithic functions; inconsistent data handling or hard-coded limits.	Spaghetti code; no modularity; violates basic systems programming principles.
Robustness (20%)	Handles complex edge cases and race conditions; perfect resource lifecycle; graceful error recovery.	Handles core features well; minor issues with fringe edge cases or synchronization logic.	Basic features work, but system is unstable; frequent resource leaks or intermittent errors.	Fails core logic; program crashes on unexpected input; incorrect usage of fundamental syscalls.
Code Engineering (10%)	No memory leaks; all syscalls check return codes; uses `perror` appropriately; safe pointer usage.	Minor leaks or missing checks on non-critical syscalls; mostly safe pointer arithmetic.	Inconsistent error handling; frequent unsafe operations; significant leaks or lack of bounds checking.	Frequent segmentation faults; silent failures of syscalls; no evidence of memory management.

Criteria	Excellent (90-100%)	Good (75-89%)	Fair (60-74%)	Poor (0-59%)
Collaboration (20%)	Professional Git usage: atomic, semantic commits; use of feature branches; evidence of team collaboration.	Consistent use of version control; adequate commit messages; evidence of a structured team workflow.	Inconsistent Git usage; large code dumps instead of incremental progress; vague commit messages.	Minimal use of version control; repository lacks history or shows no evidence of teamwork.
Technical Defense (25%)	Both members articulate the low-level mechanics; handles what-if scenarios and code-tracing confidently.	Clear architectural explanation; both members participate meaningfully; logic delivery is sound.	Unclear explanations of system mechanics; uneven participation; struggles with logic-flow questions.	Unprepared; cannot explain system flow or syscall interactions; unable to defend design decisions.

*__Remember: “Code tells you how, comments tell you why.”__ — Jeff Atwood, co-founder of Stack Overflow and Discourse